DATA STRUCTURES IMPORTANT QUESTIONS

UNIT – 2

1(a) Define Algorithm. Explain the process to find the efficiency of an algorithm with suitable examples. (4M)

Definition

An Algorithm is a finite sequence of well-defined instructions used to solve a specific problem.

Characteristics of an Algorithm

1. **Input** – Accepts zero or more inputs.
2. **Output** – Produces at least one output.
3. **Definiteness** – Each step must be clear and unambiguous.
4. **Finiteness** – Algorithm must terminate after a finite number of steps.
5. **Effectiveness** – Each step must be simple and executable.

Efficiency of an Algorithm

The **efficiency of an algorithm** is determined by analyzing the **resources it uses**, mainly:

1. **Time Complexity** – The amount of time required by the algorithm to run.
2. **Space Complexity** – The amount of memory required during execution.

Efficiency is usually expressed using **Big-O notation**.

Example

Algorithm to find the sum of first n numbers:

Algorithm

Step1: Start
Step2: Read n

Step3: sum = 0
Step4: for i = 1 to n
sum = sum + i
Step5: Print sum
Step6: Stop

The loop runs **n times**, therefore the **time complexity is O(n)**.

Conclusion

Thus, the efficiency of an algorithm is evaluated using **time complexity and space complexity** to determine how well the algorithm performs.

1(b) Describe Quick Sort Algorithm with example (Procedure, Algorithm, Program). (4M)

Definition

Quick Sort is a **divide-and-conquer sorting algorithm** that selects a **pivot element** and partitions the array around it.

Procedure

1. Select a **pivot element** from the array.
2. Rearrange elements so that
    - elements **smaller than pivot** are on the left
    - elements **greater than pivot** are on the right.
3. Recursively apply the same process to the left and right subarrays.

Algorithm

QuickSort(A, low, high)

1. if low < high
2. p = partition(A, low, high)
3. QuickSort(A, low, p-1)
4. QuickSort(A, p+1, high)

Example

Unsorted Array

8 3 7 4 9 2

Sorted Array

2 3 4 7 8 9

C Program

```
#include<stdio.h>

void quicksort(int a[], int low, int high)
{
   int i = low, j = high;
   int pivot = a[(low+high)/2], temp;

   while(i <= j)
   {
      while(a[i] < pivot) i++;
      while(a[j] > pivot) j--;

      if(i <= j)
      {
         temp = a[i];
         a[i] = a[j];
         a[j] = temp;
         i++;
         j--;
      }
   }

   if(low < j)
      quicksort(a, low, j);
   if(i < high)
      quicksort(a, i, high);
```

```
}

int main()
{
   int a[5] = {8,3,7,4,9};

   quicksort(a,0,4);

   printf("Sorted array:\n");
   for(int i=0;i<5;i++)
      printf("%d ",a[i]);

   return 0;
}
```

Conclusion

Quick sort is an efficient sorting technique with **average time complexity O(n log n)** and is widely used for sorting large datasets..

2(a) Explain Asymptotic Notations with necessary graphs. (4M)

Definition

Asymptotic notations are mathematical tools used to **describe the time complexity of an algorithm** as the input size (n) becomes very large.

They help compare the **efficiency of algorithms**.

Types of Asymptotic Notations

1. Big-O Notation (O)

- Represents the **upper bound** or **worst-case complexity** of an algorithm.
- It shows the **maximum time an algorithm may take**.

Example:
Linear search → **O(n)**

Graph representation:

Time
|
| /
| /
| /
|____/_____ n

---

2. Omega Notation (Ω)

- Represents the **lower bound** or **best-case complexity**.
- Shows the **minimum time required** by an algorithm.

Example:
Binary search best case → **Ω(1)**

Graph representation:

Time
|
|_____
| /
| /
|___/_____ n

---

3. Theta Notation (Θ)

- Represents the **tight bound** of an algorithm.
- It indicates **both upper and lower bounds**.

Example:
Merge sort → **Θ(n log n)**

Graph representation:

Time
|
| /

| /
|___/_____ n

Conclusion

Thus, **Big-O, Omega, and Theta** notations help analyze and compare algorithms based on **growth rate and efficiency**.

2(b) Describe Merge Sort Algorithm with example (Procedure, Algorithm, Program). (4M)

Definition

Merge Sort is a **divide-and-conquer sorting algorithm** that divides the array into smaller parts, sorts them, and then merges them.

Procedure

1. Divide the array into **two halves**.
2. Recursively sort the **left and right halves**.
3. **Merge the sorted halves** to produce a sorted array.

Algorithm

MergeSort(A, low, high)

1. if low < high
2. mid = (low + high)/2
3. MergeSort(A, low, mid)
4. MergeSort(A, mid+1, high)
5. Merge(A, low, mid, high)

Example

Unsorted array

8 3 7 4

Step-1 (Divide)

8 3 7 4

Step-2 (Divide again)

8 3 7 4

Step-3 (Merge)

3 8 4 7

Final sorted array

3 4 7 8

C Program

```
#include <stdio.h>

void merge(int a[], int low, int mid, int high)
{
  int i=low, j=mid+1, k=low, temp[20];

  while(i<=mid && j<=high)
  {
    if(a[i] < a[j])
      temp[k++] = a[i++];
    else
      temp[k++] = a[j++];
  }

  while(i<=mid)
    temp[k++] = a[i++];

  while(j<=high)
    temp[k++] = a[j++];

  for(i=low;i<=high;i++)
```

```
        a[i] = temp[i];
}

void mergesort(int a[], int low, int high)
{
   int mid;
   if(low < high)
   {
      mid = (low+high)/2;
      mergesort(a, low, mid);
      mergesort(a, mid+1, high);
      merge(a, low, mid, high);
   }
}

int main()
{
   int a[5] = {8,3,7,4,2};

   mergesort(a,0,4);

   printf("Sorted array:\n");
   for(int i=0;i<5;i++)
      printf("%d ",a[i]);

   return 0;
}
```

Conclusion

Merge sort has **time complexity O(n log n)** and is efficient for **large datasets**.

3(a) Define ADT. Explain different types of ADTs with their operations. (4M)

Definition

An **Abstract Data Type (ADT)** is a logical description of a data type that defines **data and operations on the data**, without specifying how the operations are implemented.

ADT focuses on **what operations are performed**, not **how they are implemented**.

Types of ADTs

1. List ADT

A **List** is an ordered collection of elements.

Operations

- insert() – Add an element
- delete() – Remove an element
- search() – Find an element
- display() – Show elements

Example:

10 → 20 → 30 → 40

---

2. Stack ADT

A **Stack** is a linear data structure that follows **LIFO (Last In First Out)** principle.

Operations

- push() – Insert element
- pop() – Remove element
- peek() – View top element

Example

Top
|
30
20
10

---

3. Queue ADT

A **Queue** is a linear data structure that follows **FIFO (First In First Out)** principle.

Operations

- enqueue() – Insert element
- dequeue() – Remove element
- front() – Get first element

Example

Front → 10 20 30 40 ← Rear

Conclusion

Thus, **ADT defines data and operations conceptually**, while the actual implementation may use arrays, linked lists, or other structures.

3(b) Describe Radix Sort Algorithm with example (Procedure, Algorithm, Program). (4M)

Definition

Radix Sort is a **non-comparison sorting algorithm** that sorts numbers **digit by digit**, starting from the least significant digit.

Procedure

1. Find the **maximum number of digits** in the elements.
2. Sort numbers based on **least significant digit (LSD)**.
3. Repeat sorting for **next digits** until the most significant digit is processed.
4. After processing all digits, the array becomes sorted.

Example

Unsorted numbers

170 45 75 90 802 24 2 66

Sorted numbers

2 24 45 66 75 90 170 802

Algorithm

RadixSort(A, n)

1. Find maximum element in array
2. for digit = 1 to number_of_digits
3. perform counting sort according to digit
4. end

C Program

```
#include <stdio.h>

int getMax(int a[], int n)
{
   int max = a[0];
   for(int i=1;i<n;i++)
      if(a[i] > max)
         max = a[i];
```

```
    return max;
}

void countSort(int a[], int n, int exp)
{
    int output[n], count[10]={0};

    for(int i=0;i<n;i++)
        count[(a[i]/exp)%10]++;

    for(int i=1;i<10;i++)
        count[i]+=count[i-1];

    for(int i=n-1;i>=0;i--)
    {
        output[count[(a[i]/exp)%10]-1]=a[i];
        count[(a[i]/exp)%10]--;
    }

    for(int i=0;i<n;i++)
        a[i]=output[i];
}

void radixSort(int a[], int n)
{
    int m=getMax(a,n);

    for(int exp=1; m/exp>0; exp*=10)
        countSort(a,n,exp);
}

int main()
{
    int a[]={170,45,75,90,802,24,2,66};
    int n=8;
```

```
    radixSort(a,n);

    printf("Sorted array:\n");
    for(int i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}
```

Conclusion

Radix sort sorts numbers **digit by digit** and works efficiently for **large numbers with fixed digits**.

4(a) Describe Merge Sort Algorithm with suitable example (Procedure, Algorithm, Program). (4M)

Definition

Merge Sort is a **divide and conquer sorting algorithm** that divides the array into smaller subarrays, sorts them, and then merges them to produce a sorted array.

Procedure

1. Divide the array into **two halves**.
2. Recursively sort the **left half** and **right half**.
3. Merge the two sorted halves to obtain the final sorted array.

Algorithm

MergeSort(A, low, high)

1. If low < high
2. mid = (low + high) / 2
3. MergeSort(A, low, mid)
4. MergeSort(A, mid + 1, high)
5. Merge(A, low, mid, high)

Example

Unsorted array

8 3 7 4

Step-1 (Divide)

8 3 7 4

Step-2 (Divide again)

8 3 7 4

Step-3 (Merge)

3 8 4 7

Final sorted array

3 4 7 8

C Program

```
#include <stdio.h>

void merge(int a[], int low, int mid, int high)
{
   int i=low, j=mid+1, k=low, temp[20];

   while(i<=mid && j<=high)
   {
      if(a[i] < a[j])
         temp[k++] = a[i++];
```

```
    else
        temp[k++] = a[j++];
}

while(i<=mid)
    temp[k++] = a[i++];

while(j<=high)
    temp[k++] = a[j++];

    for(i=low;i<=high;i++)
        a[i] = temp[i];
}

void mergesort(int a[], int low, int high)
{
    int mid;
    if(low < high)
    {
        mid = (low + high)/2;
        mergesort(a, low, mid);
        mergesort(a, mid+1, high);
        merge(a, low, mid, high);
    }
}

int main()
{
    int a[5] = {8,3,7,4,2};

    mergesort(a,0,4);

    printf("Sorted array:\n");
    for(int i=0;i<5;i++)
        printf("%d ",a[i]);
```

```
    return 0;
}
```

Conclusion

Merge sort is efficient for large datasets with **time complexity O(n log n)**.

4(b) Define Algorithm. Explain the process to find the efficiency of an algorithm with suitable examples. (4M)

Definition

An **Algorithm** is a finite sequence of well-defined steps used to solve a problem.

Characteristics of an Algorithm

1. **Input** – Takes zero or more inputs.
2. **Output** – Produces at least one output.
3. **Definiteness** – Steps must be clearly defined.
4. **Finiteness** – Must terminate after finite steps.
5. **Effectiveness** – Each step must be simple and executable.

Efficiency of an Algorithm

The efficiency of an algorithm is determined by:

1. **Time Complexity** – Time taken to execute the algorithm.
2. **Space Complexity** – Memory required by the algorithm.

Efficiency is expressed using **Big-O notation**.

Example

Algorithm to find the sum of first **n numbers**:

Step 1: Start
Step 2: Read n
Step 3: sum = 0
Step 4: for i = 1 to n
    sum = sum + i
Step 5: Print sum
Step 6: Stop

The loop runs **n times**, so the **time complexity is O(n)**.

Conclusion

Thus, algorithm efficiency is measured using **time complexity and space complexity** to evaluate performance.